

PetClinic

Sistema de Gestion Veterinaria

Integrante: Michael Galindo

Documento de Arquitectura y Base de Datos

Junio 2026

PetClinic - Sistema de Gestion Veterinaria

Documentacion Completa del Proyecto

1. ARQUITECTURA GENERAL DEL SISTEMA

1.1 Que es PetClinic?

PetClinic es un sistema completo de gestion para veterinarias que permite administrar dueos, mascotas, turnos (citas) e historial clinico. El sistema esta compuesto por un backend API, un frontend web y un modulo de automatizacion de pruebas.

1.2 Estructura del Proyecto

El repositorio contiene las siguientes carpetas principales:

- backend/ -> API REST con Python FastAPI
- frontend-web/ -> Aplicacion web construida con React 19
- app-movil/ -> Carpeta vacia - aplicacion movil pendiente de implementar
- PetClinicAutomation/ -> Suite de automatizacion con Serenity BDD + Cucumber
- DOCUMENTACION/ -> Diagramas y documentacion del proyecto

1.3 Stack Tecnologico

Componente	Tecnologia	Version	Estado
Backend	Python FastAPI	>0.110.0	Activo
Frontend	React	19.2.7	Activo
Base Datos	MySQL	-	Activo
ORM Python	SQLAlchemy	>2.0.0	Activo
Automatizacion	Serenity BDD	5.3.9	Activo
App Movil	-	-	Pendiente

2. MODELO DE DOMINIO (BASE DE DATOS)

2.1 Entidades del Sistema

El sistema maneja 5 entidades principales que representan las tablas de la base de datos MySQL:

Entidad	Tabla BD	Campos Principales
Dueno	duenos	cedula (PK), nombre, telefono
Mascota	mascotas	id (PK), nombre, especie, edad, dueno_cedula (FK)
Turno	turnos	id (PK), fecha, motivo, mascota_id (FK)
Historial	historial_clinico	id (PK), fecha, descripcion, diagnostico, mascota_id (FK)
Usuario	usuarios	id (PK), username, password_hash, rol, dueno_cedula (FK)

2.2 Relaciones entre Entidades

Las relaciones entre entidades son las siguientes:

- Dueno (1) ---- (*) Mascota: Un dueo puede tener muchas mascotas
- Mascota (1) ---- (*) Turno: Una mascota puede tener muchos turnos/citas
- Mascota (1) ---- (*) HistorialClinico: Una mascota puede tener muchos registros clinicos
- Dueno (1) ---- (1) Usuario: Un dueo tiene un usuario asociado para login

2.3 Diagrama de Relaciones (Mermaid)

Dueno (1) --- (*) ---> Mascota (1) --- (*) ---> Turno
Mascota (1) --- (*) ---> HistorialClinico
Dueno (1) --- (1) ---> Usuario

3. BACKEND - PYTHON FASTAPI

3.1 Arquitectura en Capas

El backend principal sigue un patron de arquitectura en capas:

- Router Layer (endpoints): Maneja las peticiones HTTP y retorna respuestas JSON
- CRUD Layer (datos): Contiene todas las operaciones de base de datos
- Model Layer (entidades): Define las tablas con SQLAlchemy ORM
- Schema Layer (validacion): Define los DTOs con Pydantic para validacion

3.2 A

Archivo	Funcion
main.py	Punto de entrada, configura CORS, incluye routers, crea tablas
database.py	Conexion a MySQL, crea engine y sesion, get_db()
models.py	Modelos SQLAlchemy: Dueño, Mascota, Turno, Historial, Usuario
schemas.py	Schemas Pydantic para validacion de request/response
crud.py	Todas las operaciones CRUD de cada entidad
security.py	Hash bcrypt, creacion y verificacion JWT
auth_dependencies.py	Dependencia para extraer usuario del token
roles.py	Dependencia require_role() para autorizacion por rol

3.3 Endpoints de la API

Metodo	Ruta	Descripcion
POST	/auth/login	Iniciar sesion, retorna token JWT
POST	/auth/register	Registrar nuevo usuario
POST	/auth/register-client	Registrar cliente + dueo
GET	/api/duenos	Listar dueos (admin: todos, user: propio)
POST	/api/duenos	Crear nuevo dueo
PUT	/api/duenos/{cedula}	Actualizar dueo
DELETE	/api/duenos/{cedula}	Eliminar dueo (solo admin)
GET	/api/mascotas	Listar mascotas
POST	/api/mascotas	Crear mascota
PUT	/api/mascotas/{id}	Actualizar mascota
DELETE	/api/mascotas/{id}	Eliminar mascota (solo admin)
GET	/api/turnos	Listar turnos
POST	/api/turnos	Crear turno
PUT	/api/turnos/{id}	Actualizar turno

PetClinic - Sistema de Gestion Veterinaria

Documentacion Completa del Proyecto

DELETE	/api/turnos/{id}	Eliminar turno (solo admin)
GET	/api/historiales	Listar historial clinico
POST	/api/historiales	Crear registro clinico (solo admin)
PUT	/api/historiales/{id}	Actualizar registro (solo admin)
DELETE	/api/historiales/{id}	Eliminar registro (solo admin)
GET	/api/dashboard/stats	Estadisticas del dashboard

PetClinic - Sistema de Gestion Veterinaria

Documentacion Completa del Proyecto

3.4 Dependencias Python (requirements.txt)

Libreria	Version	Proposito
fastapi	>=0.110.0	Framework web async para APIs REST
uvicorn[standard]	>=0.28.0	Servidor ASGI para ejecutar FastAPI
sqlalchemy	>=2.0.0	ORM para mapear objetos a tablas MySQL
pymysql	>=1.1.0	Driver para conectarse a MySQL
cryptography	>=42.0.0	Requerido por pymysql para auth MySQL
pydantic	>=2.6.0	Validacion de datos y serializacion JSON
python-jose[cryptography]	-	Creacion y verificacion de tokens JWT
passlib[bcrypt]	-	Hashing de contraseñas con algoritmo bcrypt

4. FRONTEND - REACT 19

4.1 Estructura del Proyecto

El frontend es una SPA (Single Page Application) construida con React 19 usando Create React App. Utiliza React Router v7 para navegacion y manejo de rutas basado en roles.

4.2 Archivos Principales

Archivo	Funcion
App.js	Componente raiz: router, layout con sidebar, estado de auth
index.css	Sistema de diseno global: tokens CSS, reset, tipografia, botones
services/api.js	Capa HTTP centralizada con fetch, headers JWT, manejo respuestas
pages/Login/Login.js	Formulario dual login/registro, decodifica JWT
pages/Dashboard/Dashboard.js	Panel admin con tarjetas de estadisticas
pages/Duenos/Duenos.js	CRUD completo de dueños con tabla y formulario inline
pages/Mascotas/Mascotas.js	CRUD completo de mascotas
pages/Turnos/Turnos.js	CRUD completo de turnos con picker de fecha
pages/ClientPortal/ClientPortal.js	Portal del cliente con tabs: mascotas, turnos, historial

4.3 Rutas por Rol

Rol	Rutas Disponibles	Redirect por Defecto
Sin login	Todas renderizan Login	-
ADMIN	/dashboard, /duenos, /mascotas, /turnos	/dashboard
USER	/portal	/portal

4.4 Manejo de Estado

No se usa libreria externa de estado (Redux, Zustand). Todo se maneja con:

- useState: Estado local en cada componente (listas, formularios, loading)
- useEffect: Carga de datos al montar componentes, llamadas a la API
- localStorage: Persistencia de token JWT y datos del usuario
- App.js: Estado raiz con isLoggedIn y user (nombre, rol, cedula)

4.5 S

Tema oscuro personalizado con CSS Custom Properties:

- Color primario: Teal (#14b8a6)
- Color secundario: Violet (#8b5cf6)
- Fondo: #0a0f1e, #111827, #1a2035
- Fuente: Inter (Google Fonts)
- Responsive: Media query en 768px para movil

5. SISTEMA DE AUTENTICACION Y SEGURIDAD

5.1 Flujo de Login

1. El usuario ingresa username y password en el formulario Login
2. Se envia POST a /auth/login con las credenciales
3. El backend verifica el password con bcrypt (hash)
4. Si es correcto, genera un token JWT con payload: sub(username), rol, dueno_cedula
5. El frontend guarda el token en localStorage
6. Se decodifica el JWT para extraer rol y cedula del usuario
7. Todas las peticiones HTTP incluyen el header: Authorization: Bearer [token]

5.2 JWT (JSON Web Token)

- Algoritmo: HS256 (HMAC-SHA256)
- Expiracion: 60 minutos
- Payload contiene: sub (usuario), rol (ADMIN/USER), dueno_cedula
- Secret key: petclinic-secret-key (hardcoded)

5.3 R

Operacion	ADMIN	USER
Crear Dueño	Permitido	Permitido
Crear Mascota	Permitido	Permitido
Crear Turno	Permitido	Permitido
Ver Dashboard	Permitido	Permitido
Ver Sus Propios Datos	Permitido	Permitido
Eliminar Dueño	Permitido	No permitido
Eliminar Mascota	Permitido	No permitido
Eliminar Turno	Permitido	No permitido
Crear Historial	Permitido	No permitido
Eliminar Historial	Permitido	No permitido

5.4 CORS (Cross-Origin Resource Sharing)

Configurado para permitir peticiones desde:

- http://localhost:3000 (desarrollo frontend)
- http://localhost:5173 (Vite dev server)
- http://127.0.0.1:3000 y :5173
- https://pet-clinic-oywb.vercel.app (produccion Vercel)

6. AUTOMATIZACION DE PRUEBAS - SERENITY BDD

6.1 Stack de Testing

- Framework BDD: Serenity BDD 5.3.9 + Cucumber 7.15.0
- Browser Automation: Selenium WebDriver (via Serenity Screenplay)
- Driver Management: WebDriverManager 5.6.3
- Test Runner: JUnit 4.13.2
- Navegador: Chrome (default), Edge (opcional)

6.2 P

El proyecto usa el patron Screenplay que separa:

- Models (DuenoData, MascotaData, TurnoData): DTOs para datos de prueba
 - Tasks (7 acciones): AbrirPetClinic, LoginPetClinic, RegistrarDueno, RegistrarMascota, RegistrarTurno, EliminarMascota, EliminarDueno
- Questions (5 validaciones): ValidarDueno, ValidarMascota, ValidarTurno, ValidarEliminacion, ValidarDashboard
- User Interfaces (5 page objects): LoginPage, DashboardPage, DuenosPage, MascotasPage, TurnosPage
- Utils: DatosPrueba (factory con datos aleatorios para evitar duplicados)

6.3 E

#	Escenario	Que Prueba
1	Registrar un dueño exitosamente	CRUD de dueño con cedula unica
2	Registrar mascota con dueño existente	Mascota Firulais ligada a cedula 10001
3	Registrar turno con mascota existente	Cita con motivo "Vacunacion anual"
4	Consultar mascotas registradas	Navegar a /mascotas, verificar titulo
5	Consultar turnos registrados	Navegar a /turnos, verificar titulo
6	Dashboard muestra contadores	Verificar tarjetas de estadísticas
7	Editar dueño existente	Cambiar telefono de cedula 10001
8	Cerrar sesion del sistema	Logout y verificar redirect a login

6.4 Como Ejecutar

Comando: gradle clean test aggregate

Reporte: target/site/serenity/index.html (reporte HTML unico)

7. BASE DE DATOS - MYSQL

7.1 Configuracion

- Motor: MySQL
- Nombre de base de datos: veterinaria
- Host: localhost:3306
- Usuario: root
- Password: Admin123*
- DDL Mode: update (Hibernate crea/actualiza tablas automaticamente)

7.2 S

Las tablas se crean automaticamente via SQLAlchemy. Schema de la base de datos:

```
CREATE TABLE duenos (  
    cedula VARCHAR(20) PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    telefono VARCHAR(20)  
);  
  
CREATE TABLE mascotas (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    especie VARCHAR(50),  
    edad INT,  
    dueno_cedula VARCHAR(20),  
    FOREIGN KEY (dueno_cedula) REFERENCES duenos(cedula)  
);  
  
CREATE TABLE turnos (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    fecha DATETIME NOT NULL,  
    motivo TEXT,  
    mascota_id INT,  
    FOREIGN KEY (mascota_id) REFERENCES mascotas(id)  
);  
  
CREATE TABLE historial_clinico (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    fecha DATETIME NOT NULL,  
    descripcion TEXT,  
    diagnostico TEXT,  
    mascota_id INT,  
    FOREIGN KEY (mascota_id) REFERENCES mascotas(id)  
);  
  
CREATE TABLE usuarios (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    username VARCHAR(50) UNIQUE NOT NULL,  
    password_hash VARCHAR(255) NOT NULL,  
    rol VARCHAR(20) DEFAULT 'USER',  
    dueno_cedula VARCHAR(20),  
    FOREIGN KEY (dueno_cedula) REFERENCES duenos(cedula)  
);
```

7.3 Indices y Restricciones

- PRIMARY KEY: cedula (duenos), id (mascotas, turnos, historial, usuarios)
- FOREIGN KEY: mascotas -> duenos, turnos -> mascotas, historial -> mascotas, usuarios -> duenos
- UNIQUE: username en usuarios

PetClinic - Sistema de Gestion Veterinaria

Documentacion Completa del Proyecto

- NOT NULL: nombre, fecha, username, password_hash
- DEFAULT: rol = USER en usuarios

8. HERRAMIENTAS Y CONFIGURACION

8.1 Control de Versiones

- Git: Sistema de control de versiones local
- GitHub: Repositorio remoto para colaboracion
- VS Code / IntelliJ IDEA: Editores de codigo
- Postman: Testing manual de endpoints API
- draw.io: Creacion de diagramas UML (clases)
- MySQL Workbench: Administracion de base de datos
- Frontend: Vercel (deploy automatico desde GitHub)
- Backend: Servidor local en http://localhost:8080
- Script de inicio: run_backend.bat (ejecuta uvicorn)

8.2 H

8.3 D

8.4 A

Archivo	Funcion
backend/requirements.txt	Dependencias Python del backend
frontend-web/package.json	Dependencias y scripts npm (React)
frontend-web/vercel.json	Reglas de reescritura SPA para Vercel
serenity.conf	Configuracion de navegador y reports

9. RESUMEN EJECUTIVO

PetClinic es un sistema de gestion veterinaria que implementa las siguientes funcionalidades:

Funcionalidades Implementadas

- Registro y gestion de duenos de mascotas
- Registro y gestion de mascotas (especie, edad, dueo)
- Gestion de turnos/citas veterinarias
- Historial clinico de cada mascota
- Autenticacion con JWT y roles (ADMIN/USER)
- Dashboard con estadisticas (total duenos, mascotas, turnos)
- Portal del cliente para ver sus datos
- 8 escenarios de prueba automatizados (E2E)

Tecnologias Clave

- Backend: Python FastAPI + SQLAlchemy + JWT + MySQL
- Frontend: React 19 + React Router v7 + CSS Custom Properties
- Testing: Serenity BDD + Cucumber + Selenium (Screenplay Pattern)
- Base de Datos: MySQL con 5 tablas y relaciones 1:N

Estado del Proyecto

Componente	Estado
Backend FastAPI	COMPLETO - CRUD, Auth, Dashboard
Frontend React	COMPLETO - Login, CRUD, Portal, Dashboard
Base de Datos MySQL	COMPLETO - 5 tablas, relaciones OK
Automatizacion E2E	COMPLETO - 8 escenarios, reportes HTML
App Movil	PENDIENTE - Carpeta vacia
Documentacion	PARCIAL - Diagrama de clases UML